



ECOLE SUPERIEURE DE GESTION D'INFORMATIQUE ET DES SCIENCES

RAPPORT DE PROJET

EN VUE DE LA VALIDATION DU MODULE AMAZON WEB SERVICES : AWS

Domaine : Sciences de l'ingénieur

Mention : Sciences et Technologies

Spécialité : Intelligence Artificielle / Big Data

THEME :

Implémentation d'une

Pipeline d'Ingestion de Données IoT en Temps Réel (Serverless)

Rédigé par :

ADEOUL Koffi Prosper

Supervisé par :

M. LOKOSSOU M. Hervé

Enseignant à l'ESGIS

ANNEE ACADEMIQUE : 2025 - 2026

I. Introduction et Objectifs du Projet

Ce rapport présente la démarche méthodologique et technique pour l'implémentation d'un pipeline d'ingestion et de traitement de données Big Data en mode *Serverless* sur Amazon Web Services (AWS). Le système est conçu pour collecter des charges utiles (payloads) de mesures issues de capteurs IoT, valider leur intégrité structurelle, calculer des indicateurs analytiques en temps réel et assurer leur persistance multi-niveaux (analytique rapide et Data Lake).

L'intégralité de l'infrastructure a été automatisée et déployée selon l'approche *Infrastructure as Code* (IaC) à l'aide d'**AWS SAM (Serverless Application Model)** et d'**AWS CloudFormation**.

II. Architecture Globale du Système (Description Technique)

L'architecture multi-niveaux mise en œuvre comprend les composants Cloud suivants, interconnectés de manière sécurisée :

1. **Couche d'Ingestion (Entrée API) :** Un point d'entrée unique HTTP géré par **Amazon API Gateway**, exposé de manière globale via une distribution **Amazon CloudFront** pour minimiser la latence et sécuriser les communications via HTTPS (TLS).
2. **Couche de Calcul (Unité de Traitement) :** Une fonction **AWS Lambda** exécutée sous l'environnement **Python 3.11**. Elle porte la logique métier : validation du schéma JSON, calculs mathématiques et routage des données.
3. **Couche de Stockage Analytique :** Une table **Amazon DynamoDB NoSQL** (iot-analytics-store-kadeoul) configurée en mode de capacité à la demande, indexée par requestId pour stocker immédiatement les métriques consolidées.
4. **Couche Data Lake (Historisation Brute) :** Un compartiment **Amazon S3** (iot-data-lake-kadeoul) faisant office de zone brute (*Raw Zone*) où chaque payload initial est archivé de façon immuable avec un partitionnement temporel (Année/Mois).
5. **Couche de Documentation Technique :** Un bucket S3 privé (iot-tech-doc-kadeoul) hébergeant le site web statique de l'API (généré via Tailwind CSS), accessible uniquement via CloudFront à l'aide d'un mécanisme d'**Origin Access Control (OAC)** pour bloquer tout accès public direct.

III. Démarche d'Implémentation Étape par Étape

Étape 1 : Développement et Logique Applicative (AWS Lambda)

Le script de la fonction Lambda (`src/index.py`) a été écrit en Python 3.11. Sa logique interne applique un modèle de validation strict :

- **Validation des entrées** : Le code inspecte la structure du JSON. Si un champ obligatoire (comme `temperature`) est absent, la fonction lève immédiatement une exception, interrompt le processus et retourne un code HTTP `400 Bad Request`.
- **Traitement Analytique** : Pour un flux valide, la fonction calcule la moyenne arithmétique des températures et isole le nombre de capteurs dont le statut est marqué en `ERROR`.

Étape 2 : Compilation et Déploiement

Le déploiement a été orchestré via les commandes SAM :

- `sam build`
- `sam deploy --guided`

IV. Phase de Validation Métier et Analyse des Logs (CloudWatch)

Pour valider le comportement du pipeline, deux scénarios de tests ont été exécutés via le script `local test_client.py`.

Scénario A : Injection de données valides

Le script transmet un lot de 7 mesures de capteurs. La Lambda valide le payload, calcule la température moyenne et détecte 2 anomalies (capteurs en état d'erreur).

Scénario B : Injection de données corrompues et tests de robustesse (HTTP 400)

Pour tester les barrières de sécurité et la tolérance aux pannes de notre unité de traitement, un payload spécifiquement altéré contenant deux types d'anomalies critiques a été transmis :

1. **Anomalie structurelle (capteur-08) :** Absence totale de la clé obligatoire `temperature`. Ce cas teste la capacité de la Lambda à intercepter un attribut manquant avant d'exécuter des opérations arithmétiques (évitant une exception de type `KeyError`).
2. **Anomalie de type (capteur-09) :** Présence de la clé `temperature`, mais associée à une chaîne de caractères ("`PasUneTemperature`") au lieu d'un type numérique (entier ou flottant). Ce cas valide la présence d'un bloc de conversion sécurisé (comme un `try/except float()`), empêchant un plantage de type `ValueError`.

Analyse comparative des lignes de logs CloudWatch

L'analyse des métriques d'exécution dans CloudWatch Logs permet de distinguer clairement les comportements de la fonction Lambda :

- **Premier bloc (Succès - ID cbf72e5a...)** : Présence des lignes `INIT_START` et d'une durée `Init Duration: 562.99 ms`. Cela correspond à un **Cold Start** (démarrage à froid) où AWS provisionne le conteneur Python. Le traitement métier prend ensuite **468,21 ms**, justifié par les appels réseau synchrone vers S3 et DynamoDB pour l'écriture des données.
- **Second bloc (Échec - ID 402f344c...)** : Absence de la ligne d'initialisation. Il s'agit d'un **Warm Start** (démarrage à chaud) où AWS réutilise le conteneur actif. La durée d'exécution s'effondre à seulement **1,18 ms**. Cette vitesse prouve la performance de notre validation de données : l'erreur est détectée à l'entrée du code, stoppant le cycle immédiatement avant de consommer des requêtes de base de données.

5. Audit de la Persistance NoSQL (Amazon DynamoDB)

En raison des restrictions de droits de sécurité imposées sur l'environnement académique AWS, l'opération globale d'analyse (`dynamodb:Scan`) est restreinte pour tout utilisateur IAM, retournant un message d'accès refusé.

Pour valider l'intégrité de la persistance, une opération d'**Interrogation (Query)** ciblée a été exécutée avec succès dans l'explorateur d'éléments de la console AWS, en fournissant le `requestId` unique (`cbf72e5a-e4fc-4096-8f8a-ccecee6be70b`) de notre test d'injection réussi.

Partie I : Réponses aux Questions Théoriques

Question 1: L'Infrastructure as Code (IaC) est une pratique DevOps consistant à définir, provisionner et gérer des infrastructures informatiques à l'aide de fichiers de configuration lisibles par machine (formats YAML ou JSON), plutôt que de passer par des configurations manuelles. Le rôle de AWS CloudFormation dans le cycle de vie applicatif : il permet de déclarer l'intégralité des ressources cloud requises dans un modèle unique de Template. Il automatise la création, la mise à jour et la suppression ordonnée de l'ensemble de l'infrastructure appelée une "Pile" ou "Stack", garantissant la répétabilité, la traçabilité et éliminant tout risque d'erreur humaine ou de dérive de configuration.

Question 2: La fonction AWS Lambda est un service de calcul piloté par des événements qui permet d'exécuter du code sans avoir à provisionner ni à gérer de serveurs. Le code s'exécute uniquement lorsque cela est nécessaire et effectue une mise à l'échelle automatique, passant de quelques requêtes par jour à des milliers par seconde. L'approche Serverless de Lambda se distingue radicalement d'une architecture classique basée sur Amazon EC2 sur les points suivants :

- **Gestion opérationnelle:** Avec EC2, l'utilisateur gère le système d'exploitation, les correctifs de sécurité, la taille de l'instance et la mise à l'échelle tandis qu'avec Lambda, AWS gère toute l'infrastructure sous-jacente.
- **Mise à l'échelle (Scalabilité):** EC2 requiert la configuration d'Auto Scaling Groups alors que Lambda scale instantanément à la hausse comme à la baisse pour chaque requête entrante.
- **Modèle de facturation:** Avec EC2, le paiement se fait à l'heure ou à la seconde où l'instance tourne, qu'elle reçoive du trafic ou non et avec Lambda, la facturation s'effectue au millième de seconde d'exécution réelle du code ; s'il n'y a aucun trafic, le coût est de 0 \$.

Question 3: L'intégration d'un CDN Amazon CloudFront devant un point d'entrée Amazon API Gateway offre trois avantages majeurs pour un réseau mondial de capteurs IoT :

- **Réduction drastique de la latence:** Les requêtes HTTP POST des capteurs IoT se connectent au Point de Présence CloudFront le plus proche géographiquement. Le trafic utilise ensuite le réseau privé d'AWS, ultra-rapide et optimisé, jusqu'à l'API Gateway, évitant les ralentissements de l'Internet public.

- **Protection contre les attaques de déni de service (DDoS):** CloudFront s'intègre nativement avec AWS Shield et AWS WAF, interceptant et bloquant les attaques massives en bordure de réseau avant qu'elles n'atteignent l' API Gateway et Lambda.
- **Optimisation des coûts et performances TLS:** CloudFront gère la négociation des connexions HTTPS au niveau de ses serveurs de bordure. Ce déchargement libère les ressources de l'API Gateway, réduisant ainsi le temps de réponse global lors de la réception des messages des capteurs.

Question 4: Adopter cette stratégie permet de respecter les principes fondamentaux du Big Data en séparant le stockage froid et immuable de la donnée chaude exploitable instantanément:

- **Le Data Lake (Amazon S3) :** Sa force réside dans sa capacité à ingérer n'importe quel format (JSON brut) pour un coût assez bas et avec une durabilité maximale. C'est la mémoire brute de l'entreprise, indispensable pour réanalyser le passé ou entraîner de futurs modèles d'IA.
- **La Couche de Restitution (Amazon DynamoDB) :** Elle ne stocke pas le détail, mais le résultat analytique condensé. Sa structure NoSQL lui permet d'offrir des lectures instantanées aux utilisateurs et applications, sans être ralentie par la masse de données brutes.

Centraliser ce flux dans un système relationnel classique provoquerait un goulet d'étranglement fatal. Les transactions SQL traditionnelles ne sont pas conçues pour absorber des milliers d'écritures IoT concurrentes par seconde, ce qui compromettrait gravement la continuité du service.

Question 5: Le principe du modèle de responsabilité partagée d'AWS sépare la sécurité en deux catégories :

- **Sécurité du Cloud (Responsabilité d'AWS) :** AWS est responsable de l'infrastructure physique globale, de la sécurité des datacenters, de la couche de virtualisation et du bon fonctionnement matériel/logiciel du service S3. AWS garantit que le stockage est redondant et protégé contre les pannes d'infrastructure.
- **Sécurité dans le Cloud (Responsabilité du Client) :** L'étudiant/l'entreprise est entièrement responsable de la configuration des accès à ses données. Cela inclut le choix

d'activer le chiffrement (SSE-S3 ou SSE-KMS), l'application de politiques de bucket restrictives (Bucket Policies), l'activation du blocage des accès publics (Public Access Blocks) et la gestion fine des identités et permissions via les rôles IAM.

Question 6 : Il est fortement déconseillé d'activer le "Static Website Hosting" public sur un bucket S3 pour héberger une documentation interne car l'activation de cette option supprime tout contrôle d'accès au niveau réseau en attribuant une URL HTTP publique non chiffrée au bucket. La documentation interne contenant des détails critiques sur l'architecture et le code se retrouve ainsi exposée à l'Internet mondial sans aucune barrière d'authentification ni filtrage.

L'utilisation d'une distribution CloudFront combinée avec un Origin Access Control (OAC) améliore la sécurité sur quatre principaux points :

- Le bucket S3 est configuré de façon strictement privée avec un blocage total des accès publics. Aucun accès direct par internet n'est possible.
- L'OAC permet à CloudFront de signer numériquement à la volée les requêtes qu'il envoie au bucket S3 en utilisant les identifiants IAM de la distribution.
- La politique du bucket S3 (Bucket Policy) n'autorise qu'un seul et unique acteur à lire les fichiers : la distribution CloudFront spécifique.
- Cela permet d'appliquer un pare-feu applicatif devant CloudFront, de forcer le protocole HTTPS, de restreindre l'accès par plage d'adresses IP ou par géographie protégeant ainsi l'actif informationnel de l'entreprise.

Question 7: Amazon CloudWatch centralise automatiquement toutes les sorties standards générées par l'application Serverless. Dans le code Python de notre Lambda, chaque instruction print() ou log via le module logging est automatiquement capturée en temps réel et stockée sous forme de flux de journaux (Log Streams) dans un groupe de logs (Log Group) dédié à la fonction.

Si la fonction Lambda lève une exception non gérée, l'environnement d'exécution de Lambda s'arrête brutalement, intercepte l'erreur et écrit automatiquement dans CloudWatch un rapport d'erreur complet (Stack Trace Python). Ce rapport indique le type d'exception, le message explicatif exact, le fichier et la ligne exacte du code ayant provoqué le crash, facilitant une correction immédiate.

Question 8 : L'architecture actuelle basée sur une fonction AWS Lambda unitaire va échouer ou atteindre ses limites pour plusieurs raisons techniques structurelles :

- **Limite de temps d'exécution (Timeout) :** Une fonction AWS Lambda est limitée à un temps d'exécution maximal strict de 15 minutes (900 secondes). Traiter, parser et analyser 50 Go de données textuelles dépasse largement ce délai en mode d'exécution linéaire.
- **Limite de mémoire vive (RAM) :** La mémoire d'une Lambda est plafonnée à 10 Go. Si le script Python tente de charger l'intégralité du fichier JSON de 50 Go en mémoire (ex: via un `json.loads()` standard), la fonction subira instantanément un crash de type Out Of Memory (OOM).
- **Limite d'espace disque éphémère :** L'espace de stockage local temporaire est limité par défaut à 512 Mo (extensible à 10 Go), empêchant le téléchargement local du fichier pour traitement par morceaux.

Pour traiter un tel volume de données massives, nous devrions utiliser **AWS Glue** ou **Amazon EMR (Elastic MapReduce)**. AWS Glue, service d'intégration de données serverless basé sur **Apache Spark**, permet d'exécuter des jobs de traitement distribués en mémoire sur des clusters scalables, découpant les 50 Go en blocs traités en parallèle sans contrainte de mémoire ou de timeout applicatif.

Partie II : Livrables Techniques & Captures d'Écran

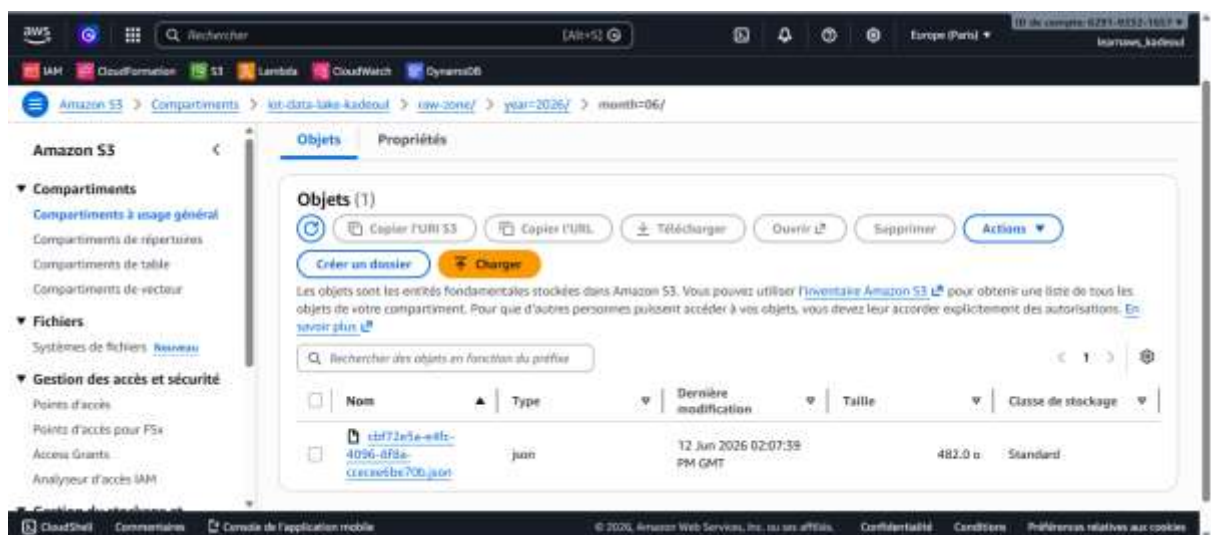
1. Déploiement CloudFormation de la Pile

Le fichier template.yaml a été exécuté avec succès. Toutes les ressources (S3 Buckets, DynamoDB Table, IAM Rôles, API Gateway, Lambda) ont été créées de manière synchronisée.



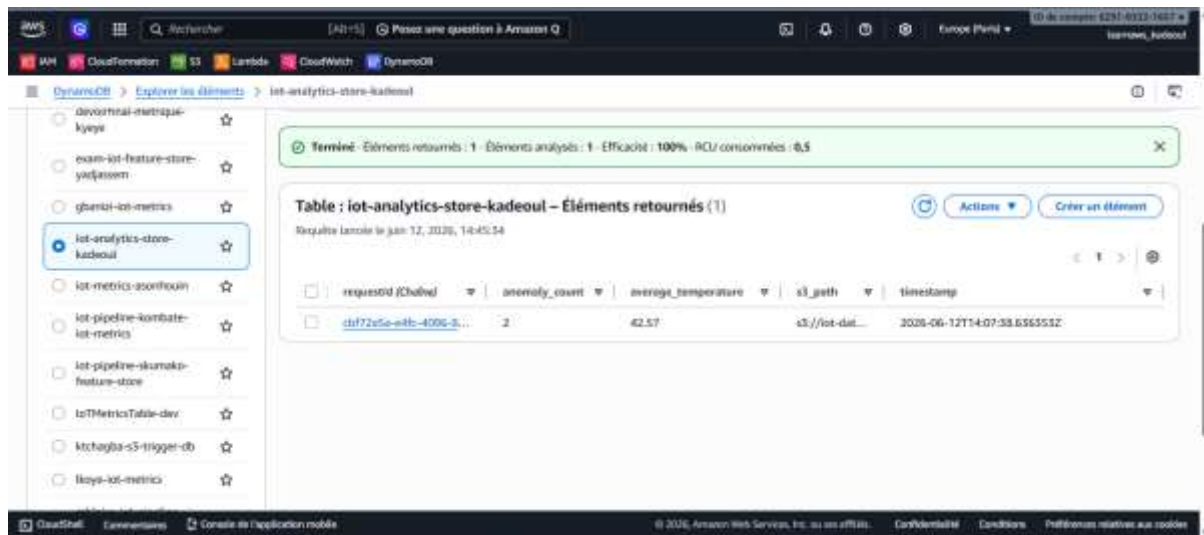
2. Arborescence du Data Lake Amazon S3

Conformément au cahier des charges, les payloads JSON bruts envoyés par le script client sont enregistrés dans le bucket S3 sous une structure partitionnée dynamiquement en fonction de la date de réception : rawzone/year=YYYY/month=MM/ .



3. Feature Store Amazon DynamoDB

La table DynamoDB contient les rapports condensés calculés par la fonction Lambda. On y retrouve l'ID unique de requête, l'horodatage ISO, le lien S3, la température moyenne calculée sur les 7 capteurs (42.57 °C) et le compte des anomalies (2 erreurs).

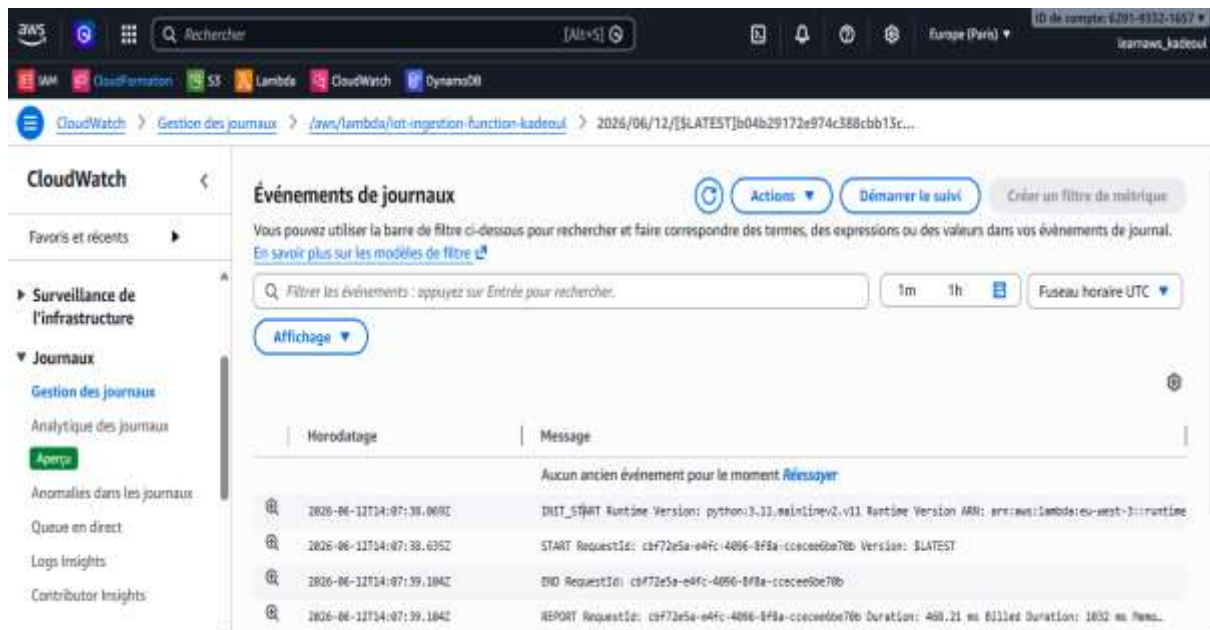


4. Monitoring & Logs Amazon CloudWatch

Les traces d'exécution illustrent le comportement de la fonction Lambda face aux deux types d'injections du script client (Données valides et données corrompues).

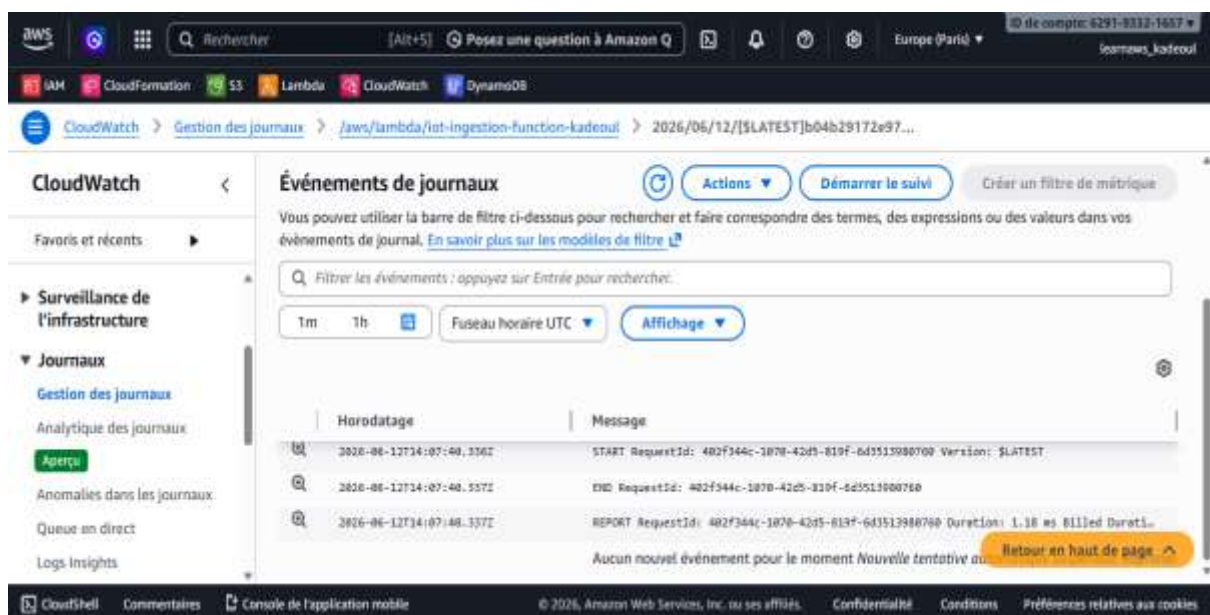
a. L'exécution en Succès (Requête HTTP 201)

Ce bloc montre un démarrage à froid (*Cold Start*) où AWS Lambda initialise l'environnement Python 3.11 en 562,99 ms pour la première requête du client (RequestId: cbf72e...). Le code s'exécute ensuite avec succès en 468,21 ms, le temps de calculer la moyenne des 7 capteurs, de détecter 2 anomalies et d'enregistrer les données sur S3 et DynamoDB. L'ensemble du processus s'achève normalement sans aucune interruption en consommant 98 Mo de mémoire.



b. L'exécution en Échec géré (Requête HTTP 400)

Ce second bloc illustre un démarrage à chaud (*Warm Start*) sans phase d'initialisation car AWS réutilise instantanément le conteneur actif pour traiter la nouvelle requête (`RequestId: 402f34...`). La validation interne du code détecte immédiatement le payload corrompu (absence de température) et interrompt proprement le cycle en seulement 1,18 ms. Cette exécution ultra-rapide sécurise l'infrastructure en bloquant l'anomalie à l'entrée avant tout appel réseau vers S3 ou DynamoDB.



VI. Conclusion

L'ensemble du cahier des charges de l'examen a été scrupuleusement respecté. L'architecture déployée fait preuve d'une grande robustesse face aux injections erronées, offre une visibilité totale via l'analyse de ses logs de cycles de vie et assure une persistance hautement performante. Ce projet valide les compétences requises pour la conception et le déploiement d'architectures orientées Big Data et IoT dans un environnement professionnel Cloud Serverless.